

Supervised Learning and Linear Regression

IN5148: Statistics and Data Science with Applications in
Engineering

Alan R. Vazquez
Department of Industrial Engineering

Agenda

1. Introduction to Supervised Learning
2. Linear Regression Model
3. K-fold Cross Validation

Introduction to Supervised Learning

Supervised Learning

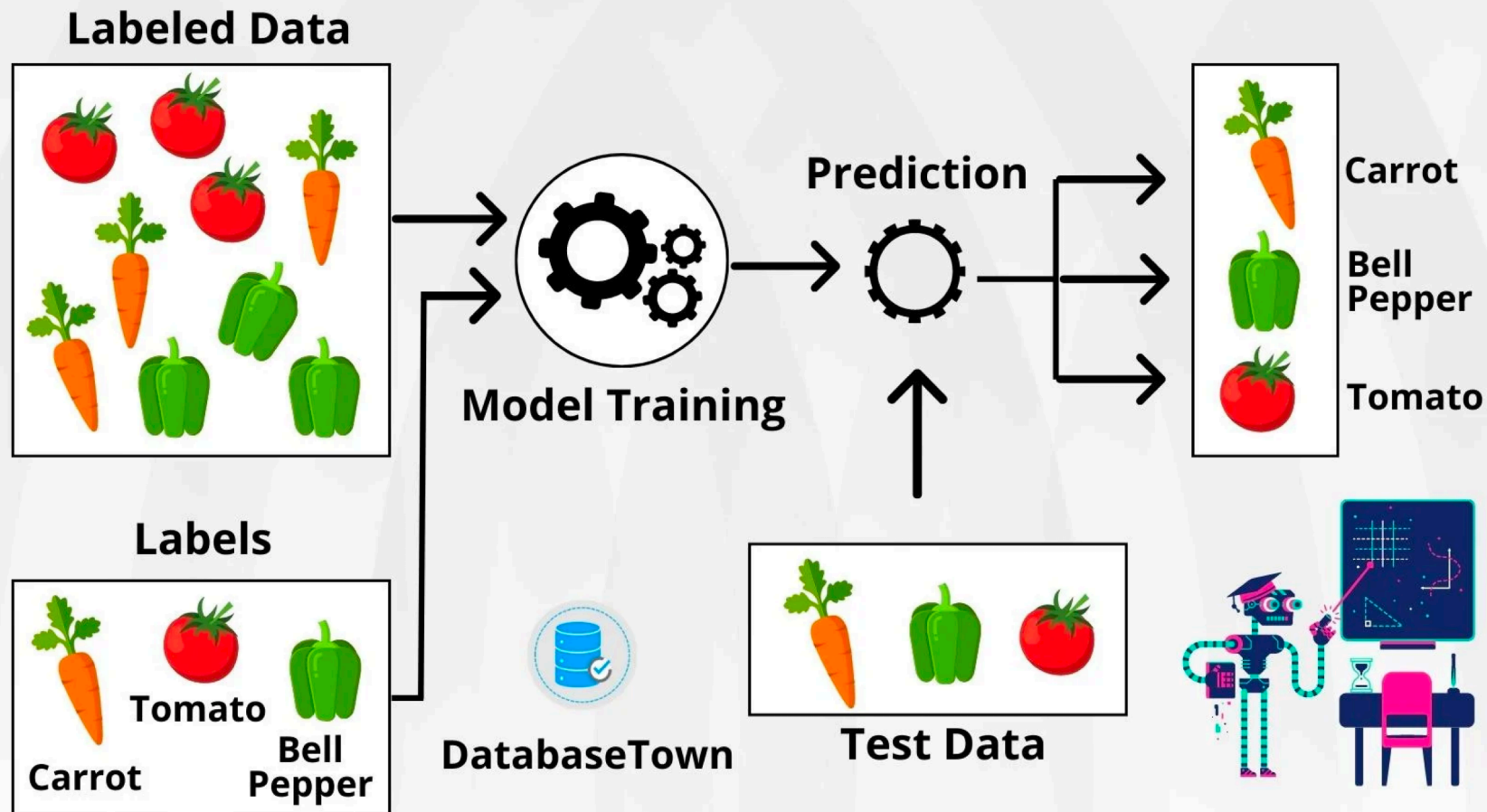
Includes algorithms that learn by example. The user provides the supervised algorithm with a known data set that includes the corresponding known inputs and outputs. The algorithm must find a method to determine how to reach those inputs and outputs.

While the user knows the correct answers to the problem, the algorithm identifies patterns in the data, learns from observations, and makes predictions.

The algorithm makes predictions that can be corrected by the user, and this process continues until the algorithm reaches a high level of accuracy and performance.

SUPERVISED LEARNING

Supervised machine learning is a branch of artificial intelligence that focuses on training models to make predictions or decisions based on labeled training data.



Supervised learning problems

Regression Problems. The response is numerical. For example, a person's income, the value of a house, or a patient's blood pressure.

Classification Problems. The response is categorical and involves K different categories. For example, the brand of a product purchased (A, B, C) or whether a person defaults on a debt (yes or no).

The predictors ($\mathbf{X}$) can be *numerical* or *categorical*.

Supervised learning problems

Regression Problems. The response is numerical. For example, a person's income, the value of a house, or a patient's blood pressure.

Classification Problems. The response is categorical and involves K different categories. For example, the brand of a product purchased (A, B, C) or whether a person defaults on a debt (yes or no).

The predictors ($\mathbf{X}$) can be *numerical* or *categorical*.

Regression problem

Goal: Find the best function $f(\mathbf{X})$ of the predictors $\mathbf{X} = (X_1, \dots, X_p)$ that describes the response Y .

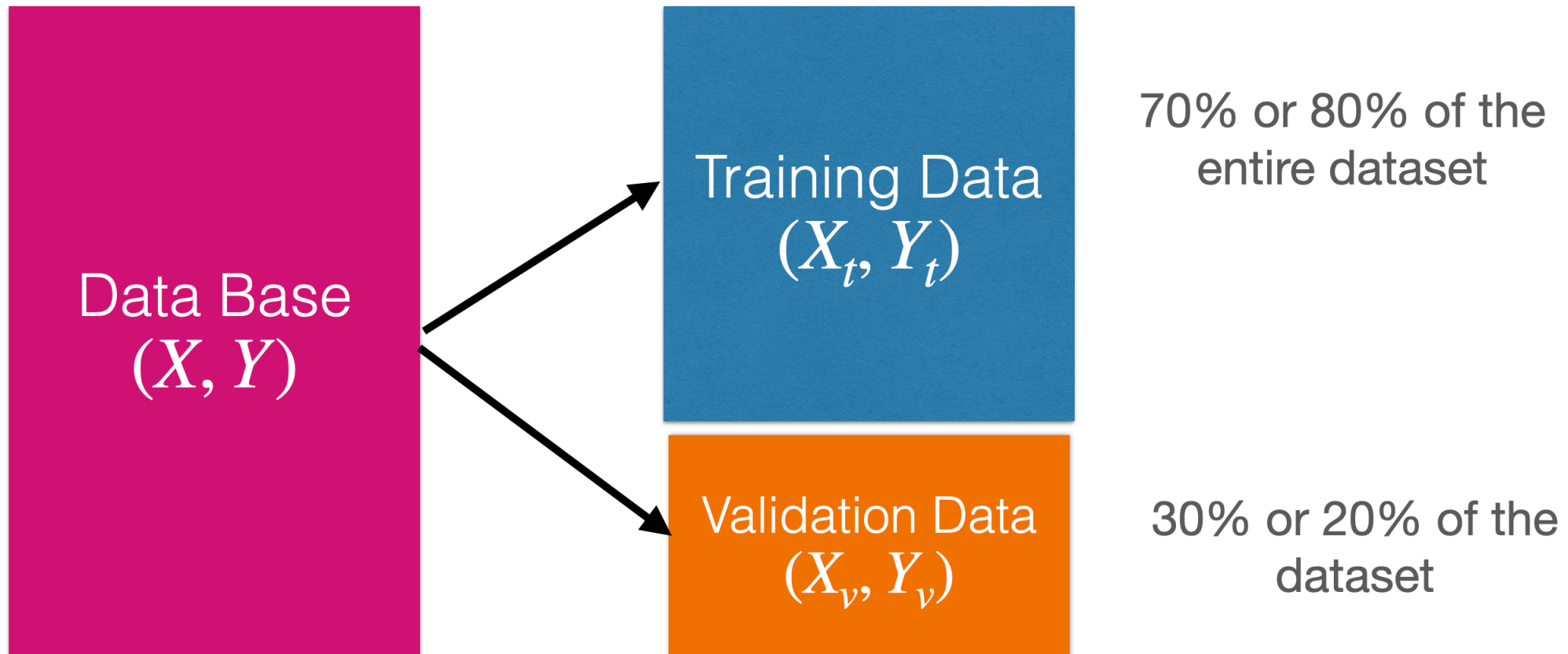
In mathematical terms, we want to establish the following relationship:

$$Y = f(\mathbf{X}) + \epsilon$$

- Where ϵ is a natural (random) error.

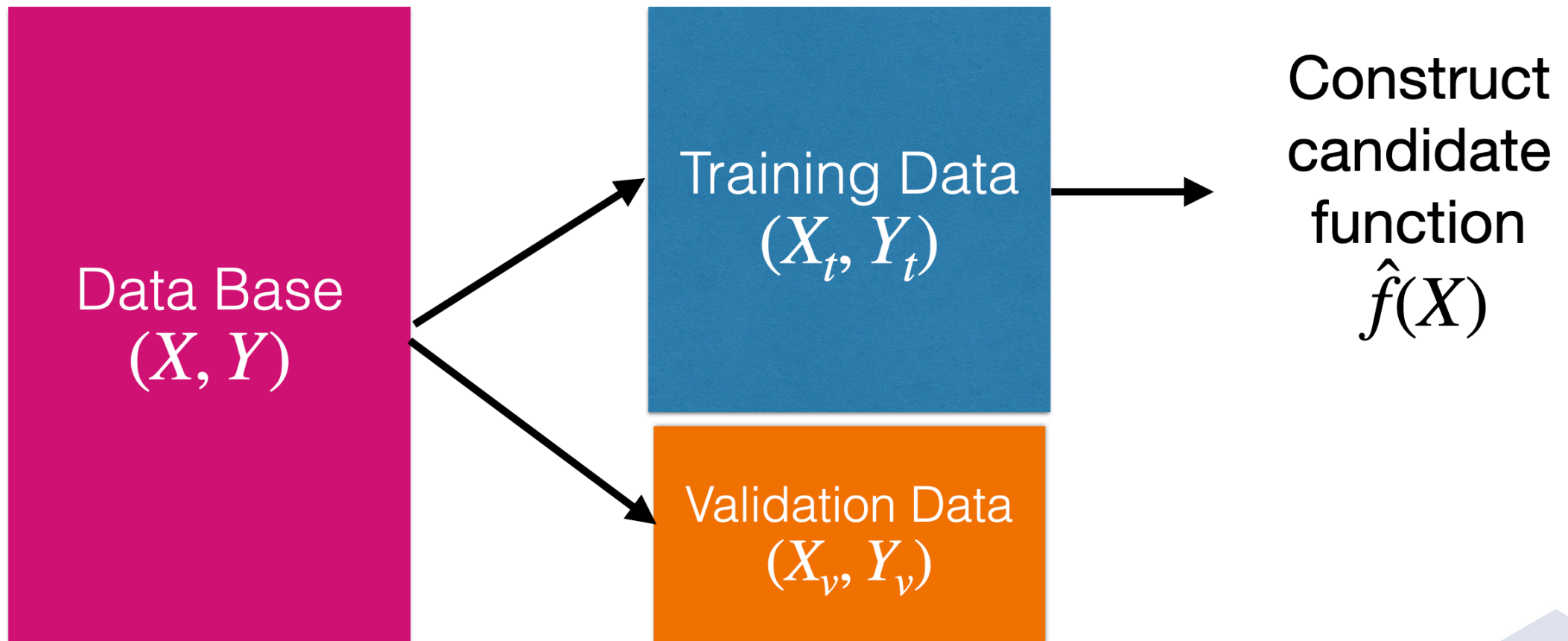
How to find the shape of $f(X)$?

Using training data.



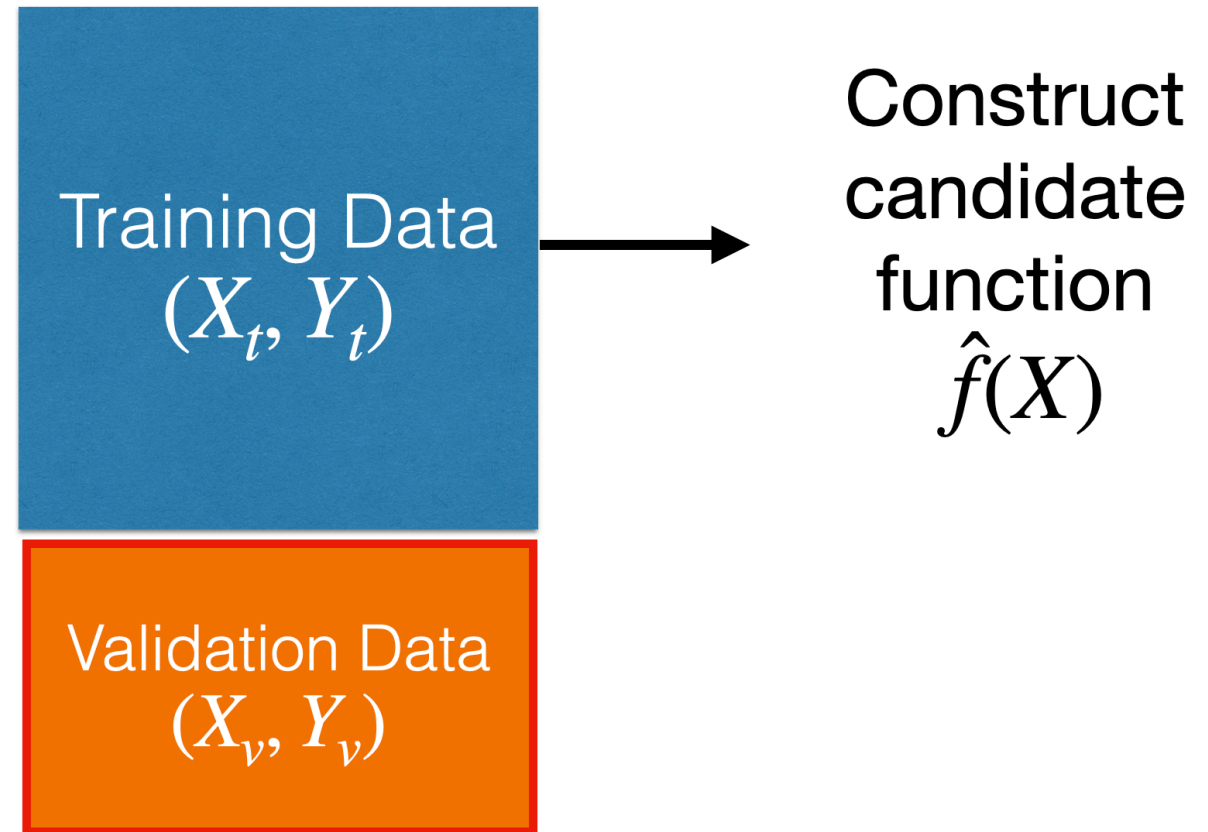
How to find the shape of $f(X)$?

Using training data.



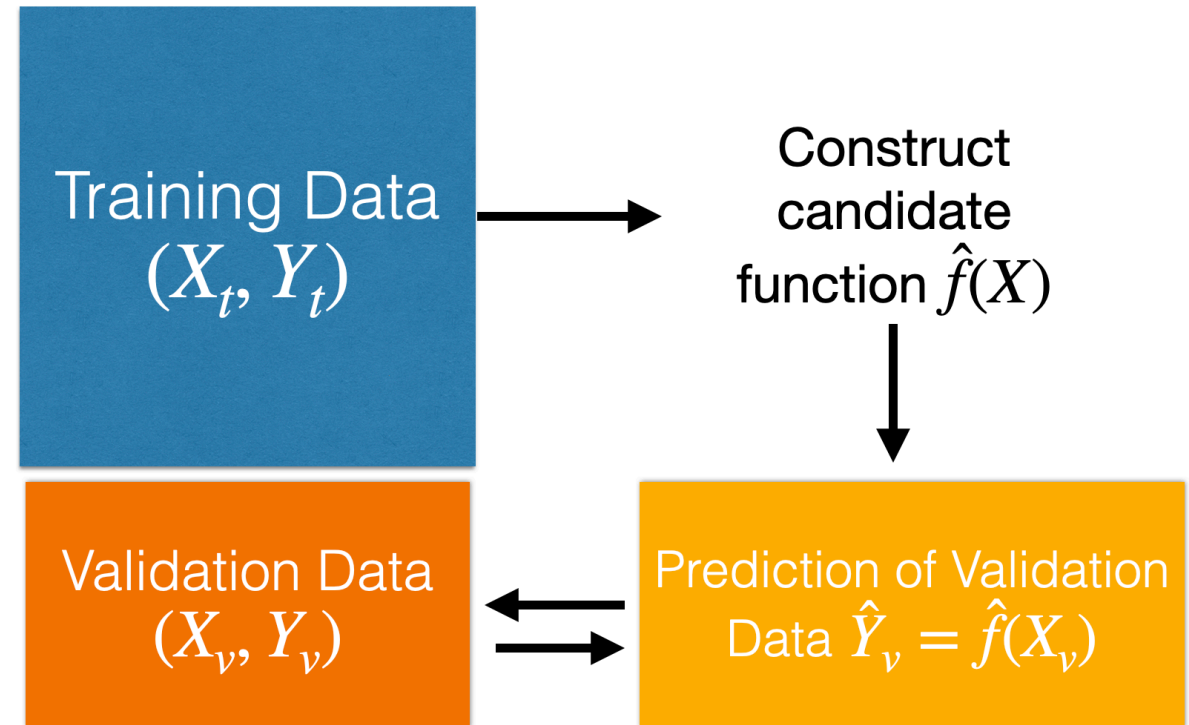
How to evaluate the quality of the candidate function $\hat{f}(X)$?

Using validation data.



How to evaluate the quality of the candidate function $\hat{f}(X)$?

Using validation data.



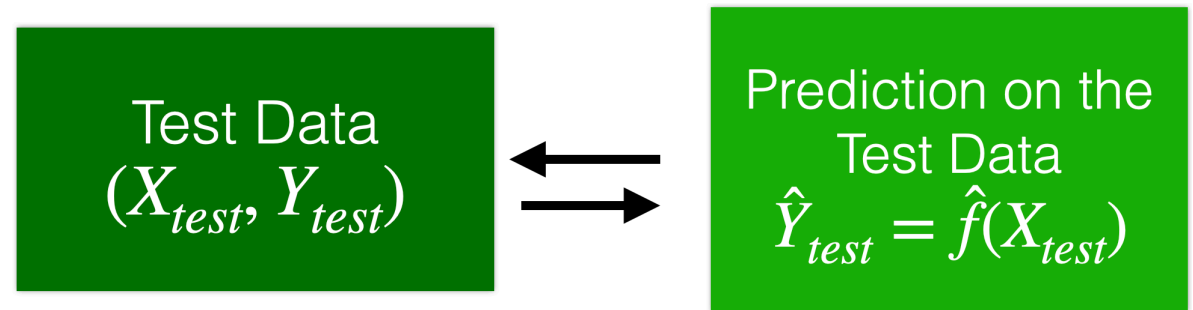
Compare the accuracy of the predictions $\hat{Y}_v$ of the real responses Y_v

Moreover...

We can use *test data* for a final evaluation of the model.

Test data is data obtained from the process that generated the training data.

Test data is independent of the training data.



Compare the accuracy of the predictions $\hat{Y}_{test}$ of the real responses Y_{test}

Linear Regression Model

Linear Regression Model

A common candidate function for predicting a response is the linear regression model. It has the mathematical form:

$$\hat{Y}_i = \hat{f}(\mathbf{X}_i) = \hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \cdots + \hat{\beta}_p X_{ip}.$$

- Where $i = 1, \dots, n_t$ is the index of the n_t training data.
- $\hat{Y}_i$ is the prediction of the actual value of the response Y_i associated with values of p predictors denoted by $\mathbf{X}_i = (X_{i1}, \dots, X_{ip})$.
- The values $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ are the *coefficients* of the model.

Interpretation of coefficients

$$\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \cdots + \hat{\beta}_p X_{ip},$$

where the unknown parameter $\hat{\beta}_0$ is called the “intercept,” and $\hat{\beta}_j$ is the “coefficient” of the j-th predictor.

For the j-th predictor, we have that:

- $\hat{\beta}_j = 0$ implies no dependence.
- $\hat{\beta}_j > 0$ implies positive dependence.
- $\hat{\beta}_j < 0$ implies negative dependence.

$$\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \cdots + \hat{\beta}_p X_{ip},$$

Interpretation:

- $\hat{\beta}_0$ is the **average response** when all predictors X_j equal 0.
- $\hat{\beta}_j$ is the **amount of increase in the average response** by a 1 unit increase in the j -th predictor X_{ij} , *when all other predictors are fixed to an arbitrary value.*

Estimation of Coefficients

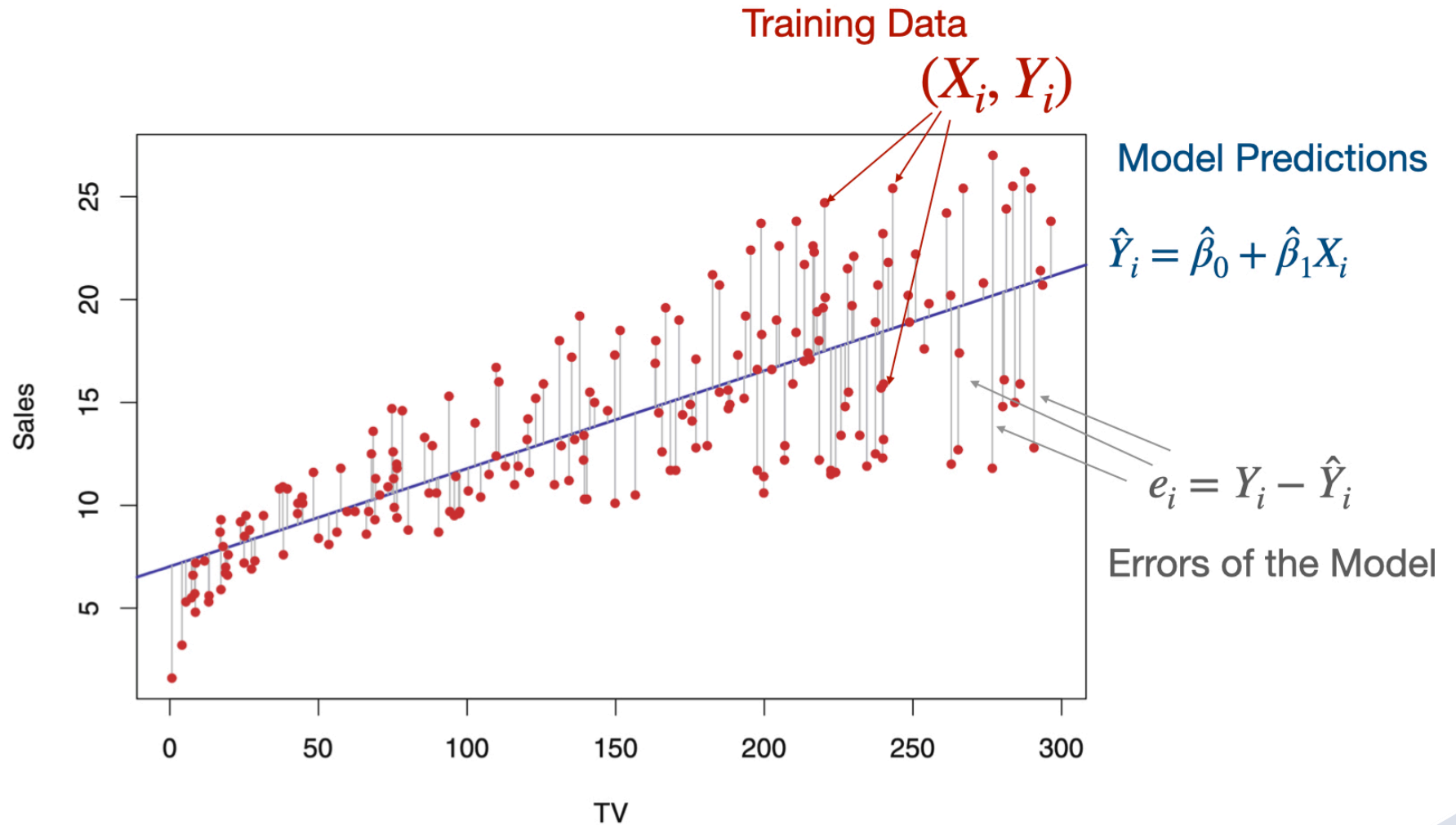
The values of $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ are obtained from the training data using method of **least squares**.

This method finds the coefficient values that minimize the error made by the model $\hat{f}(X_i)$ when trying to predict the responses in the training set:

$$RSS = \sum_{i=1}^{n_t} (Y_i - (\hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \dots + \hat{\beta}_p X_{ip}))^2$$

where RSS means *residual sum of squares*.

The idea in two dimensions



Load the libraries

Before we see an example, let's import the data science libraries into Python.

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.model_selection import train_test_split, cross_val_score
4 from sklearn.linear_model import LinearRegression
5 from sklearn.metrics import mean_squared_error, root_mean_squared_error
6 from sklearn.metrics import r2_score
```

Here, we use specific functions from the **pandas**, **matplotlib**, **seaborn** and **sklearn** libraries in Python.

Example 1

We used the dataset called “Advertising.xlsx” in Canvas.

- TV: Money spent on TV ads for a product (in hundreds of \$).
- Radio: Money spent on Radio ads for a product (in hundreds of \$).
- Newspaper: Money spent on Newspaper ads for a product (in hundreds of \$).
- Sales: Sales generated from the product (in thousands of \$).
- 200 markets

```
1 # Load the data into Python
2 Ads_data = pd.read_excel('Advertising.xlsx')
```

Notation

For our linear regression model:

- Y_i is the i -th observation for Sales.
- X_{i1} is the i -th observation for TV.
- X_{i2} is the i -th observation for Radio.
- X_{i3} is the i -th observation for Newspaper.

```
1 Ads_data.head()
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

Now, let's set the predictors and response. In the definition of `X_full`, the double bracket in `[]` is important because it allows us to have a pandas DataFrame as output. This makes it easier to fit the linear regression model with **scikit-learn**.

```
1 # Chose the predictor.  
2 X_full = Ads_data.filter(['TV', 'Radio', 'Newspaper'])  
3  
4 # Set the response.  
5 Y_full = Ads_data.filter(['Sales'])
```

Create training and validation data

To evaluate a model's performance on unobserved data, we split the current dataset into a training dataset and a validation dataset. To do this, we use the scikit-learn `train_test_split()` function.

```
1 X_train, X_valid, Y_train, Y_valid = train_test_split(X_full, Y_full,  
2                                                    test_size = 0.25,  
3                                                    random_state = 301655)
```

We use 75% of the data for training and the rest for validation.

Fit a linear regression model in Python

In Python, we use the `LinearRegression()` and `fit()` functions from the **scikit-learn** to fit a linear regression model.

```
1 # 1. Create the linear regression model
2 LRmodel = LinearRegression()
3
4 # 2. Fit the model.
5 LRmodel.fit(X_train, Y_train)
```

The following commands allow you to show the estimated coefficients of the model.

```
1 print("Coefficients:", LRmodel.coef_)
```

```
Coefficients: [[ 0.04620676  0.18682436 -0.0059342  ]]
```

We can also show the estimated intercept.

```
1 print("Intercept:", LRmodel.intercept_)
```

```
Intercept: [3.25814952]
```

The estimated model thus is

$$\hat{Y}_i = 3.258 + 0.046X_{i1} + 0.186X_{i2} - 0.005X_{i3}.$$

Interpretation

$$\hat{Y}_i = 3.258 + 0.046X_{i1} + 0.186X_{i2} - 0.005X_{i3}.$$

- The average sales are 3.258 thousands of dollars when the **advertising budgets** for TV, Radio, and Newspaper **are 0 dollars**.
- Increasing the advertising TV by 1 hundreds of dollars **increases the average sales** by 0.046 thousands of dollars.
- Increasing the advertising Radio by 1 hundreds of dollars **increases the average sales** by 0.186 thousands of dollars.
- Increasing the advertising Newspaper by 1 hundreds of dollars **reduces the average sales** by 0.005 thousands of dollars.

Prediction error

After estimating and validating the linear regression model, we can check the quality of its predictions on **unobserved** data. That is, on the data in the validation set.

One metric for this is the **mean prediction error** (MSE_v):

$$\text{MSE}_v = \frac{\sum_{i=1}^{n_v} (Y_i - (\hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \cdots + \hat{\beta}_p X_{ip}))^2}{n_v}$$

- For n_v , the validation data!

The smaller MSE_v , the better the predictions.

In practice, the square root of the mean prediction error is used:

$$\text{RMSE}_v = \sqrt{\text{MSE}_v}.$$

The advantage of RMSE_v is that it can be interpreted as:

| The average variability of a model prediction.

For example, if $\text{RMSE}_v = 1$, then a prediction of $\hat{Y} = 5$ will have an (average) error rate of ± 1 .

In Python

To evaluate the model's performance, we use the validation dataset. Specifically, we use the predictor matrix stored in `X_valid`.

In Python, we make the prediction using the pre-trained `LRmodel`.

```
1 Y_pred = LRmodel.predict(X_valid)
```


To evaluate the model, we use the function `mean_squared_error()` from **scikit-learn**. Recall that the responses from the validation dataset are in `Y_valid`, and the model predictions are in `Y_pred`.

```
1 mse = mean_squared_error(Y_valid, Y_pred) # Mean Squared Error (MSE)
2 print(round(mse, 2))
```

5.09

To obtain the **root mean squared error (RMSE)**, we use `root_mean_squared_error()` instead.

```
1 rmse = root_mean_squared_error(Y_valid, Y_pred)
2 print(round(rmse, 2))
```

2.26

Another Metric: R^2

- In the context of Data Science, R^2 can be interpreted as the *squared* correlation between the actual responses and those predicted by the model.
- The higher the correlation, the better the agreement between the predicted and actual responses.

We compute R^2 in Python as follows:

```
1 rtwo_sc = r2_score(Y_valid, Y_pred) # Rsquared  
2 print(round(rtwo_sc, 2))
```

0.78

K-fold Cross Validation

Limitations of a validation set

- By using a training data set that is much smaller than our actual data, the estimated model $\hat{f}(\mathbf{X})$ will be less good than if we used the full training data. That is, more likely for predictions to be far from actual values.
- Thus, the validation MSE is likely to be bigger than had we (a) used the full data set and (b) fit the correct model.
- In other words, using less than all data results in a MSE_v that is not a good representation of the predictive performance of $\hat{f}(\mathbf{X})$.

K -fold Cross-Validation

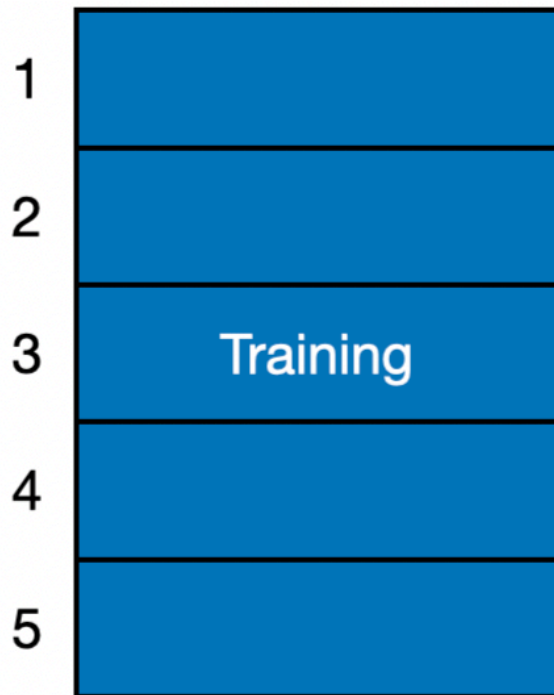
Basic Idea: Divide the training data into K equally-sized divisions or folds.



Training

K -fold Cross-Validation

Basic Idea: Divide the training data into K equally-sized divisions or folds ($K = 5$ here).



K -fold Cross-Validation

Basic Idea: Divide the training data into K equally-sized divisions or folds ($K = 5$ here).

1	Validation
2	Training
3	Training
4	Training
5	Training

- Using training, construct $\hat{f}^{(-1)}$.
- Using validation, calculate
$$CV_1(\hat{f}^{(-1)}) = \frac{1}{n_1} \sum_{i \in F_1} (Y_i - \hat{f}^{(-1)}(\mathbf{X}_i))$$
- Call it the **Fold-Based Error Estimate**

K -fold Cross-Validation

Basic Idea: Divide the training data into K equally-sized divisions or folds ($K = 5$ here).

1	Training
2	Validation
3	Training
4	Training
5	Training

- Using training, construct $\hat{f}^{(-2)}$.
- Using validation, calculate
$$CV_1(\hat{f}^{(-2)}) = \frac{1}{n_2} \sum_{i \in F_2} (Y_i - \hat{f}^{(-2)}(\mathbf{X}_i))$$

K -fold Cross-Validation

Basic Idea: Divide the training data into K equally-sized divisions or folds ($K = 5$ here).

1	Training
2	Training
3	Training
4	Training
5	Validation

- Using training, construct $\hat{f}^{(-5)}$.
- Using validation, calculate
$$CV_1(\hat{f}^{(-5)}) = \frac{1}{n_5} \sum_{i \in F_5} (Y_i - \hat{f}^{(-5)}(\mathbf{X}_i))$$

We average these fold-based error estimates to yield an evaluation metric:

$$CV(\hat{f}) = \frac{1}{K} \sum_{k=1}^K CV_k(\hat{f}^{(-k)}).$$

- Called the K -fold cross-validation estimate.
- Here, we used $K = 5$ but another popular choice is $K = 10$.

K-fold CV in Python

In Python, we apply *K*-fold cross validation (CV) using the function `cross_val_score()` from **scikit-learn**. The argument `cv` sets the number of folds to use, and `scoring` sets the evaluation metric to compute on the folds.

```
1 # 1. Define a new linear regression model
2 LRmodel_cv = LinearRegression()
3
4 # 2. Apply 5-fold CV
5 neg_cv_MSEs = cross_val_score(LRmodel_cv, X_full, Y_full, cv = 5,
6                               scoring = "neg_mean_squared_error")
7
8 # 3. Show scores
9 print(neg_cv_MSEs)
```

```
[-3.1365399  -2.42566776 -1.58522508 -5.42615506 -2.79114519]
```

Unfortunately, `cross_val_score()` outputs negative scores. We simply turn them into positive by multiplying them by -1 or adding a `-` symbol.

```
1 cv_MSEs = -neg_cv_MSEs
```

After that, we average the values using `.mean()` to obtain a the 5-fold CV estimate.

```
1 MSE_cv = cv_MSEs.mean()  
2 print(round(MSE_cv, 3))
```

```
3.073
```

Note that we can compute a K -fold CV estimate for any evaluation metric including the R^2 . To this end, we set `scoring = "r2"`.

```
1 # 1. Define a new linear regression model
2 LRmodel_cv = LinearRegression()
3
4 # 2. Apply 5-fold CV
5 cv_Rsq = cross_val_score(LRmodel_cv, X_full, Y_full, cv = 5,
6                           scoring = "r2")
7
8 # 3. Compute CV estimate
9 Rsq_cv = cv_Rsq.mean()
10 print(round(Rsq_cv, 3))
```

0.887

We can also compute a K -fold CV estimate for the root mean squared error (RMSE).

```
1 LRmodel_cv = LinearRegression()
2 neg_cv_RMSEs = cross_val_score(LRmodel_cv, X_full, Y_full, cv = 5,
3                               scoring = "neg_root_mean_squared_error")
4 cv_RMSEs = -neg_cv_RMSEs
5 RMSE_cv = cv_RMSEs.mean()
6 print(round(RMSE_cv, 3))
```

1.718

Final remarks

- K -fold CV provides a robust estimate of prediction error by averaging performance across multiple data splits.
- It improves data efficiency by allowing all observations to be used for both training and validation.
- It is widely used for **model comparison** and **hyperparameter tuning**.
- It can be extended to *generalized* K -fold cross-validation (e.g., stratified, grouped, or time-series folds) to respect data structure.

Return to main page